

BEAM: Encoding-Aware Greedy Merging for Scalable Flat Graph Summarization

Boyang Zhang
Hunan University
Changsha, Hunan, China
zhangboyang@hnu.edu.cn

Guoqing Xiao
Hunan University
Changsha, Hunan, China
xiaoguoqing@hnu.edu.cn

Yuedan Chen
Central South University
Changsha, Hunan, China
chenyuedan@hnu.edu.cn

Zhuo Tang
Hunan University
Changsha, Hunan, China
ztang@hnu.edu.cn

Xiaofei Zhang
University of Memphis
Memphis, Tennessee, USA
Xiaofei.Zhang@memphis.edu

Kenli Li
Hunan University
Changsha, Hunan, China
lkl@hnu.edu.cn

ABSTRACT

Flat correction-set graph summarization represents a graph with supernodes, superedges, and correction edges, enabling exact reconstruction with a compact flat summary. Scalable methods in this model follow an iterative divide-merge-encode pipeline, and the repeated merge stage is the main scalability challenge because candidate pairs must be generated, evaluated under the encoding objective, selected without conflicts, and applied while the summary state evolves.

We present BEAM, an encoding-aware greedy merging algorithm for scalable flat graph summarization. BEAM makes flat summarization faster by decoupling candidate discovery from objective-faithful merge scoring: lightweight profiles only prune the search space, while retained candidates are scored by exact local correction-set gain. Concretely, BEAM builds a sparse candidate graph within each divide group, applies greedy positive-gain matching within groups, and defers partition updates to the end of each group block. Retained gain evaluations are packed into batched scoring tasks, with CUDA used as one backend for that scoring step.

On ten common datasets, BEAM is faster than SWeG on all datasets and faster than SLUGGER on nine of them, with geometric mean speedups of $2.69\times$ over SWeG and $2.29\times$ over SLUGGER. BEAM also produces smaller relative output size than SWeG on all ten common datasets, while compression results against the hierarchical SLUGGER baseline remain mixed. Runtime breakdowns of the batch-ea-blocked CUDA implementation show that merge-related work accounts for 50.0%–97.9% of BEAM’s execution time, confirming that BEAM targets a major, and often dominant, cost in the flat summarization pipeline.

PVLDB Reference Format:

Boyang Zhang, Guoqing Xiao, Yuedan Chen, Zhuo Tang, Xiaofei Zhang, and Kenli Li. BEAM. PVLDB, TBD(TBD): TBD, Under review.

doi:TBD

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/githubofzby/beam>.

1 INTRODUCTION

Large graphs are now routinely used to represent web pages, social interactions, communication records, biological relations, knowledge bases, and many other forms of relational data. Directly storing and processing such graphs is expensive because graph analytics repeatedly access large, sparse, and irregular adjacency structures. A broad body of graph compression work therefore seeks to reduce storage cost through vertex ordering, reference encoding, interval encoding, locality-aware layout, recursive partitioning, and compressed graph indexes [1, 3, 5, 8, 9]. These techniques primarily encode a fixed input graph compactly. Graph summarization takes a complementary approach: it constructs a smaller structural representation that groups vertices and records the information needed to reconstruct or approximate the original graph [17].

This paper focuses on flat correction-set graph summarization, where a graph is represented by disjoint supernodes, superedges, and positive and negative correction edges [19, 21]. This model provides explicit reconstruction semantics and can serve as a compact preprocessing representation for large graphs. Scalable methods in this model commonly follow an iterative divide-merge-encode pipeline: divide restricts the search space by grouping current supernodes, merge selects beneficial supernode pairs, and encode materializes the final superedges and correction sets.

The repeated merge stage presents both a computational and an algorithmic challenge. It is executed over many iterations while the summary state is evolving. Each accepted merge changes supernode memberships, quotient neighborhoods, future candidate pairs, local pair capacities, and the encoding costs used by subsequent decisions. Consequently, the algorithm does not optimize a single static candidate graph, but a sequence of candidate graphs whose vertices,

Proceedings of the VLDB Endowment, Vol. TBD, No. TBD ISSN 2150-8097.
doi:TBD

edges, and weights are repeatedly regenerated. Exhaustively evaluating all possible pairs is infeasible, yet relying on a lightweight structural similarity alone can admit merges that do not reduce the actual correction-set cost. The central challenge is therefore to control the evolving merge-search space without allowing the search proxy to replace the encoding objective used for merge acceptance.

BEAM addresses this challenge through a simple design principle: *approximate the search space, but not the merge objective*. Lightweight profiles are used only to construct a bounded sparse candidate graph. Every retained candidate is then evaluated using its exact local correction-set gain under the current partition. Non-positive candidates are discarded, and the remaining candidate edges are processed in descending-gain order to construct a conflict-free matching. The selected merges are applied only after block-local candidate evaluation and matching have completed. BEAM thus combines profile-based candidate sparsification, exact encoding-aware gain evaluation, positive-gain matching, and blocked merge execution while preserving the standard flat correction-set representation.

This organization separates three concerns that are often entangled in iterative graph summarization. The *proposal layer* uses grouping and sparse profiles to determine which pairs receive the limited exact evaluation budget. The *decision layer* uses exact correction-set gain and conflict-free matching to determine which retained pairs are accepted. The *execution layer* maintains and updates the evolving summary state and evaluates retained candidates in batches. In our implementation, a unified exact cost oracle connects local merge gains to the corresponding global partition-cost changes, while a persistent quotient representation avoids repeatedly reconstructing current supernode neighborhoods from the original graph. Batched exact scoring can be executed by multicore CPU threads or by an optional CUDA backend; the scoring backend does not alter candidate construction, merge acceptance, matching, or partition-update semantics.

BEAM improves the merge process without changing the expressive power of the output representation. It remains within the same flat correction-set model as scalable flat methods such as SWeG [21]. In contrast, SLUGGER introduces a hierarchical representation with nested supernodes and positive and negative edges between hierarchical supernodes [14]. We therefore distinguish same-model comparisons, which evaluate different methods for constructing flat correction-set summaries, from cross-model comparisons, which provide context across representations with different expressive power.

This leads to the central research question of this work: *Can a flat graph summarizer aggressively sparsify a sequence of evolving merge-search spaces while retaining exact, encoding-aware merge decisions and well-defined blocked execution semantics?* BEAM answers this question by assigning different responsibilities to candidate discovery, merge decision, and state update: profiles control where computation is spent,

the correction-set objective controls which merges are accepted, and conflict-free blocked execution controls how accepted decisions mutate the evolving partition.

This paper makes the following contributions.

- We propose BEAM, an objective-faithful sparse merging algorithm for flat correction-set graph summarization. BEAM uses lightweight profiles to construct a bounded candidate graph, evaluates every retained pair using exact local encoding gain, selects positive-gain endpoint-disjoint pairs, and applies the selected merges in blocks.
- We formalize each merge round as a weighted matching problem on a dynamically regenerated candidate graph. We show that the gain of a retained candidate can be computed exactly from its affected local neighborhood and equals the corresponding change in global partition cost. On a fixed retained positive-gain candidate graph, BEAM’s descending-gain matching is a $1/2$ -approximation to maximum-weight matching on that graph. This guarantee is deliberately scoped to the retained graph and does not claim global optimality of the final summary.
- We provide a semantics-preserving execution substrate for the same BEAM algorithm. A unified integer cost oracle connects local gain, global partition cost, and final correction-set accounting, while a persistent quotient representation removes post-initialization reconstruction of current supernode adjacency and supports zero-copy row access. Batched CPU and CUDA scoring are interchangeable execution backends rather than separate algorithmic definitions.
- We evaluate BEAM along complementary dimensions: exact-objective and lossless-output correctness, candidate sparsification and exact-validation behavior, deterministic work eliminated by persistent state, end-to-end compression-runtime tradeoffs, and negative ablations that expose the limits of optimizing candidate counts or isolated local decisions. CUDA is evaluated separately as an optional scoring backend.

2 PROBLEM FORMULATION AND DESIGN PRINCIPLES

2.1 Flat Correction-Set Graph Summarization

Let the input be a simple undirected graph $G = (V, E)$. A flat graph summary replaces the original vertices with a partition $\mathcal{S} = \{S_1, \dots, S_r\}$, where each $S_i \subseteq V$, $S_i \cap S_j = \emptyset$ for $i \neq j$, and $\bigcup_i S_i = V$. Each set S_i is called a supernode. The partition is flat because every original vertex belongs to exactly one supernode and the summary does not maintain nested or overlapping supernodes. This representation is the basis of correction-set graph summarization and is also used by scalable flat methods such as SWeG [19, 21].

Given a flat partition, the summary stores a set of superedges P together with two correction sets C^+ and C^- . A

superedge between two supernodes represents the decision to encode the corresponding vertex-pair region densely at the summary level. Positive corrections C^+ store original edges that are not implied by the selected superedges, while negative corrections C^- store missing edges that would otherwise be implied by those superedges. We write the resulting summary as

$$\Sigma = (\mathcal{S}, P, C^+, C^-).$$

The representation has explicit reconstruction semantics: the original graph is recovered by expanding the superedges and then applying the positive and negative corrections.

For two current supernodes X and Y , let $e(X, Y)$ be the number of original graph edges represented by the pair. The number of possible undirected edges covered by the pair is

$$\text{cap}(X, Y) = \begin{cases} |X||Y|, & X \neq Y, \\ |X|(|X| - 1)/2, & X = Y. \end{cases} \quad (1)$$

The encoder has two local choices. It can represent the pair sparsely by storing the existing edges directly:

$$L_{\text{sparse}}(X, Y) = e(X, Y). \quad (2)$$

Alternatively, it can represent the pair densely by storing one superedge and recording the missing edges as negative corrections:

$$L_{\text{dense}}(X, Y) = 1 + \text{cap}(X, Y) - e(X, Y), \quad (3)$$

where the constant term accounts for the superedge itself. The local correction-set cost is therefore

$$L(X, Y) = \min \{L_{\text{sparse}}(X, Y), L_{\text{dense}}(X, Y)\}. \quad (4)$$

For a fixed partition $\mathcal{S}$, we define its exact correction-set cost as the sum over all unordered supernode pairs, including self-pairs:

$$\text{Cost}(\mathcal{S}) = \sum_{X \in \mathcal{S}} L(X, X) + \sum_{\{X, Y\} \in \binom{\mathcal{S}}{2}} L(X, Y). \quad (5)$$

Here, $\binom{\mathcal{S}}{2}$ denotes the unordered pairs of distinct supernodes. Equation (5) is the minimum correction-set cost of the fixed partition under the pairwise sparse-or-dense encoding rule in Equation (4). It is also the objective according to which BEAM evaluates candidate merges.

BEAM changes neither this objective nor the expressive power of the representation. Its output remains a flat partition with superedges and correction edges rather than a hierarchical summary. Its algorithmic task is to search efficiently over partitions induced by successive supernode merges.

Table 1 summarizes the notation used throughout the paper.

2.2 Evolving Divide-and-Merge Construction

Flat correction-set summaries are commonly constructed through an iterative divide-and-merge pipeline. Starting from an initial partition, the algorithm repeatedly groups currently active supernodes, searches for promising pairs inside each

Table 1: Frequently used notation.

Symbol	Meaning
$G = (V, E)$	Input graph.
$\mathcal{S}$	Current flat partition into supernodes.
Σ	Flat summary $(\mathcal{S}, P, C^+, C^-)$.
X, Y, Z	Current supernodes.
P	Superedge set in the final flat summary.
C^+, C^-	Positive and negative correction sets.
W_X	Sparse neighborhood profile of supernode X .
Q	Divide group of active supernodes.
C_Q	Retained candidate-pair set inside group Q .
C_Q^+	Positive-gain edges retained from C_Q .
M_{XY}	Merged supernode $X \cup Y$.
$\mathcal{S} \downarrow (X, Y)$	Partition after merging X and Y .
$\mathcal{M}_Q$	Conflict-free merge set selected inside group Q .
$\text{cap}(X, Y)$	Number of possible original edges represented by pair (X, Y) .
$L(X, Y)$	Local correction-set cost for pair (X, Y) .
$\text{Cost}(\mathcal{S})$	Exact correction-set cost of partition $\mathcal{S}$.
$\text{gain}(X, Y)$	Exact encoding gain of merging X and Y .

group, evaluates whether merging a pair reduces the encoding cost, and applies accepted merges to update the partition.

SWeG follows this general strategy and explicitly separates dividing, merging, and encoding: shingle-based grouping restricts the candidate search space, the merging step greedily considers within-group pairs, and the final encoding step materializes superedges and corrections [21]. In such a pipeline, a cheap structural proxy is useful because evaluating the true correction-set gain for every possible pair is too expensive. SWeG therefore uses SuperJaccard to choose a candidate partner inside each group and then applies a saving threshold to decide whether to accept the merge [21].

Later scalable summarizers follow the same broad divide-and-merge idea but alter either the candidate-search mechanism or the output representation. LDME-style methods and related scalable summarizers use locality-sensitive hashing to find promising merges efficiently [20, 27]. SLUGGER also reduces candidate search, but changes the underlying representation to a hierarchical model and performs greedy merges under that model [14]. BEAM instead keeps the flat correction-set representation fixed and studies how candidate search, objective evaluation, conflict resolution, and partition updates should be organized within that model.

The difficulty is not merely that a single candidate set may be large. The search problem evolves after every accepted block of merges. A merge changes supernode memberships, quotient neighborhoods, pair capacities, future candidate edges, and the gains assigned to those edges. The algorithm therefore operates over a sequence of dynamically regenerated candidate graphs rather than one static weighted graph. Exhaustive exact evaluation of every possible pair is

infeasible, but allowing a lightweight search proxy to determine merge acceptance can detach the resulting trajectory from the actual correction-set objective.

A key limitation of similarity-based or proxy-based search is that the proxy is not itself the encoding objective. A profile score measures whether two supernodes have related neighborhood patterns, whereas the actual compression effect depends on supernode sizes, local pair capacities, edge counts, affected neighboring supernodes, and changes between sparse and dense encoding choices. A highly ranked pair may therefore produce no encoding reduction, while a moderately ranked pair may yield positive gain because savings accumulate over several affected pair costs.

As a simple running example, first consider two singleton supernodes A and B . Suppose each has one edge to Z_1 and one edge to Z_2 , where Z_1 and Z_2 each contain at least two vertices, and assume there are no internal edges and no edge between A and B . A Jaccard-style profile similarity would rank A and B highly because their external profiles are identical. However, the external cost before merging is

$$L(A, Z_1) + L(B, Z_1) + L(A, Z_2) + L(B, Z_2) = 1 + 1 + 1 + 1 = 4.$$

After merging A and B , each merged pair is still encoded sparsely:

$$L(A \cup B, Z_1) = 2, \quad L(A \cup B, Z_2) = 2.$$

The external cost remains 4, and the endpoint and self-pair terms do not change under the assumptions above. Thus, this highly ranked pair has zero encoding gain.

Conversely, consider two supernodes C and D , each of size 3, and four neighboring supernodes $Z_1, \dots, Z_4$, each also of size 3. Suppose C has five edges to each of Z_1, Z_2, Z_3 , while D has five edges to each of Z_1, Z_2, Z_4 , again with no internal edges and no edge between C and D . Their profiles overlap only on Z_1 and Z_2 , so their profile similarity is moderate. Yet for each shared neighbor Z_i , the pre-merge cost is

$$L(C, Z_i) + L(D, Z_i) = 5 + 5 = 10,$$

whereas after merging C and D , the dense encoding cost becomes

$$L(C \cup D, Z_i) = 1 + 18 - 10 = 9.$$

The merge saves one unit on each shared neighbor, and these savings accumulate over the affected neighborhood.

This example motivates the central design principle of BEAM: lightweight profiles determine which pairs receive exact evaluation, but they do not certify or accept merges. Every retained candidate is validated using the correction-set objective defined above.

2.3 Exact Local-to-Global Encoding Gain

Consider a candidate merge (X, Y) under the current partition $\mathcal{S}$. Let

$$M_{XY} = X \cup Y$$

be the merged supernode, and denote the resulting partition by

$$\mathcal{S} \downarrow (X, Y) = (\mathcal{S} \setminus \{X, Y\}) \cup \{M_{XY}\}. \quad (6)$$

Only encoding terms incident to X or Y can change under this partition transformation. All other supernode-pair costs are identical in $\mathcal{S}$ and $\mathcal{S} \downarrow (X, Y)$ and cancel when the cost difference is computed. We define the affected scope as

$$\Omega(X, Y) = \{X, Y\} \cup \{Z \in \mathcal{S} : e(X, Z) > 0 \text{ or } e(Y, Z) > 0\}. \quad (7)$$

The external affected neighborhood excludes the two merge endpoints:

$$N_{XY} = \Omega(X, Y) \setminus \{X, Y\}. \quad (8)$$

Before merging X and Y , the affected terms contribute

$$L_{\text{before}}(X, Y) = L(X, X) + L(Y, Y) + L(X, Y) + \sum_{Z \in N_{XY}} (L(X, Z) + L(Y, Z)). \quad (9)$$

After replacing X and Y by M_{XY} , the corresponding terms contribute

$$L_{\text{after}}(X, Y) = L(M_{XY}, M_{XY}) + \sum_{Z \in N_{XY}} L(M_{XY}, Z). \quad (10)$$

The exact local encoding gain is

$$\text{gain}(X, Y) = L_{\text{before}}(X, Y) - L_{\text{after}}(X, Y). \quad (11)$$

The locality of Equation (11) is not an approximation. It gives exactly the change in the global partition objective:

$$\text{gain}(X, Y) = \text{Cost}(\mathcal{S}) - \text{Cost}(\mathcal{S} \downarrow (X, Y)). \quad (12)$$

To see this, partition the terms in Equation (5) into those incident to X or Y and all remaining terms. The latter terms are unchanged by the merge. The former terms are exactly $L_{\text{before}}(X, Y)$ before the merge and $L_{\text{after}}(X, Y)$ afterward.

A positive value of $\text{gain}(X, Y)$ therefore means that applying this merge in isolation strictly decreases the current flat correction-set cost. BEAM uses this exact value as the weight of a retained candidate edge and excludes non-positive candidates before matching.

Equation (12) concerns one merge evaluated against one partition state. When several endpoint-disjoint merges are scored against a frozen state and later committed together, their isolated gains need not sum exactly to the realized batch-level cost change. In particular, pair costs between the endpoints of two different selected merges may change jointly after both merges are applied. BEAM therefore treats exact local gain as an objective-faithful edge weight for block-local selection, not as a certificate of global optimality for the final summary.

2.4 Retained Candidate Graph and Merge Selection

Consider one divide group Q . Candidate discovery produces a deduplicated retained set

$$C_Q \subseteq \binom{Q}{2}.$$

Each retained pair $\{X, Y\} \in C_Q$ is evaluated under the current partition using Equation (11). The positive-gain edge set is

$$C_Q^+ = \{\{X, Y\} \in C_Q : \text{gain}(X, Y) > 0\}. \quad (13)$$

These edges induce a weighted candidate graph

$$H_Q = (Q, C_Q^+, w), \quad w(X, Y) = \text{gain}(X, Y).$$

Given this fixed retained graph, the block-local merge-selection subproblem is

$$\begin{aligned} \max_{\mathcal{M} \subseteq C_Q^+} \quad & \sum_{\{X, Y\} \in \mathcal{M}} w(X, Y) \\ \text{s.t.} \quad & \text{each } X \in Q \text{ is incident to at most one edge in } \mathcal{M}. \end{aligned} \quad (14)$$

Thus, conflict resolution on a fixed retained graph is a maximum-weight matching problem. Exact maximum-weight matching is polynomial-time solvable [10]. In BEAM, however, Equation (14) is only a repeatedly generated local subproblem. After selected merges are applied, the supernodes, candidate edges, and edge weights change, and candidate generation and exact evaluation must be performed again.

BEAM therefore processes the edges of C_Q^+ in descending gain order and greedily accepts an edge when neither endpoint has already been used. This produces a deterministic conflict-free matching while avoiding the cost of repeatedly solving exact weighted matching on many small, dynamically regenerated candidate graphs. On a fixed retained positive-gain graph, the standard charging argument gives a 1/2-approximation to maximum-weight matching on that same graph. The guarantee is deliberately restricted to the conflict-resolution step after candidate sparsification and exact gain evaluation; it is not an approximation guarantee for the globally optimal flat summary.

2.5 Algorithmic Separation and Execution Boundary

The preceding formulation separates three responsibilities in the repeated merge process.

First, the *proposal layer* controls the search budget. It groups the current supernodes and uses lightweight structural profiles to construct the retained candidate sets C_Q . Proposal quality affects which portions of the evolving merge space are explored, but a proxy score is not used as a correctness certificate.

Second, the *decision layer* assigns every retained edge its exact correction-set gain, removes non-positive edges, and constructs a conflict-free matching on the resulting weighted candidate graph. This layer determines which proposed merges are accepted.

Third, the *execution layer* applies the selected merges and maintains the partition state from which the next sequence of candidate graphs will be generated. BEAM uses blocked execution: the candidates in a block are prepared, scored, and matched against a frozen partition state, and the selected endpoint-disjoint merges are committed after block-local selection has completed. This boundary keeps candidate discovery, objective evaluation, conflict resolution, and state mutation conceptually separate.

The separation also exposes backend-neutral parallelism. Once a retained pair and the required local statistics have been prepared, its exact gain can be evaluated independently

of other retained pairs against the same frozen partition state. Retained-pair evaluations can therefore be packed into batched scoring units and processed by multicore CPU workers or by an optional CUDA backend. The surrounding candidate construction, conflict resolution, and evolving-state updates retain irregular control and mutation dependencies of the kind commonly encountered in parallel graph workloads [24].

Backend selection changes only how the exact weights of retained candidate edges are evaluated. It does not change the candidate set, the correction-set objective, positive-gain filtering, matching policy, or blocked merge semantics. CUDA is consequently an optional scoring backend within BEAM rather than part of the definition of the BEAM algorithm.

3 BEAM ALGORITHM

3.1 Overview

BEAM takes as input a simple undirected graph $G = (V, E)$ and produces a flat correction-set summary $\Sigma = (\mathcal{S}, P, C^+, C^-)$ under the objective defined in Section 2.

BEAM follows an iterative divide-and-merge construction, but assigns different responsibilities to candidate proposal, merge decision, and state mutation. In each iteration, BEAM first divides the current active supernodes into disjoint groups. Within each group, it uses sparse structural profiles to construct a bounded candidate graph. Every retained candidate edge is then assigned its exact correction-set gain under the current partition. After removing non-positive edges, BEAM processes the resulting weighted graph in descending-gain order to obtain an endpoint-disjoint matching. Selected merges are accumulated within a group block and committed only after all groups in that block have completed selection.

The central distinction is that BEAM sparsifies the evolving merge search space but does not approximate the objective used to accept a retained pair. The profile-based score controls which pairs receive exact evaluation; the exact gain determines which retained edges are eligible for matching. Greedy matching approximates conflict resolution only on the resulting retained positive-gain graph. Blocked application then defines when the selected decisions mutate the partition.

More concretely, in iteration t , the divide stage produces disjoint groups

$$\mathcal{Q}_t = \{Q_1, \dots, Q_r\}.$$

The groups are partitioned into group blocks

$$\mathcal{B}_t = \{B_1, \dots, B_s\}.$$

At the beginning of each block B , BEAM freezes the current partition state as $\mathcal{S}_B$. All candidate profiles, retained edges, and exact gains for groups in B are evaluated against $\mathcal{S}_B$. Because the divide groups are disjoint, the group-local matchings can be combined into one endpoint-disjoint block merge set. The block is committed only after these group-local decisions have completed.

Algorithm 1: BEAM Algorithm

Input: graph $G = (V, E)$; iteration budget T ; candidate budget k .

Output: flat correction-set summary $\Sigma = (S, P, C^+, C^-)$.

```
1: Initialize  $S \leftarrow \{\{v\} : v \in V\}$ .
2: Initialize the active supernode set  $A \leftarrow S$ .
3: for  $t = 1$  to  $T$  do
4:    $Q_t \leftarrow \text{ADAPTIVEMULTIHASHDIVIDE}(A)$ .
5:   Partition  $Q_t$  into group blocks  $\mathcal{B}_t$ .
6:   for each group block  $B \in \mathcal{B}_t$  do
7:      $S_B \leftarrow S$  // frozen block state
8:      $\mathcal{M}_B \leftarrow \emptyset$ .
9:     for each group  $Q \in B$  do
10:       $\mathcal{M}_Q \leftarrow \text{GREEDYGAINMATCHING}(Q, k, S_B)$ .
11:       $\mathcal{M}_B \leftarrow \mathcal{M}_B \cup \mathcal{M}_Q$ .
12:     end for
13:      $S \leftarrow \text{APPLYMERGEBATCH}(S, \mathcal{M}_B)$ .
14:      $A \leftarrow \text{Active}(S)$ .
15:   end for
16: end for
17:  $(P, C^+, C^-) \leftarrow \text{ENCODEFLATSUMMARY}(G, S)$ .
18: return  $\Sigma = (S, P, C^+, C^-)$ .
```

Figure 1: Overall BEAM procedure. Candidate construction, exact gain evaluation, and matching within a block use the same frozen partition state. The resulting endpoint-disjoint merges are committed at block granularity.

Algorithm 1 gives the overall procedure. The algorithmic description is independent of the data structure used to maintain the partition and the backend used to evaluate retained edges. Section ?? describes semantics-preserving execution strategies for these operations.

3.2 Profile-Based Candidate Sparsification

Candidate sparsification controls the amount of exact merge evaluation. It determines which parts of the current merge space BEAM explores, but does not determine which retained pairs are accepted.

At the beginning of each iteration, BEAM partitions the active supernodes into disjoint divide groups. Candidate discovery is restricted to pairs inside the same group rather than performed over all active supernodes. Within each group, BEAM constructs sparse profiles under the current partition and uses an encoding-aware proxy to retain at most k candidate partners per source supernode.

The result for group Q is the deduplicated unordered candidate set C_Q introduced in Section 2.4. Only pairs in C_Q proceed to exact gain evaluation.

3.2.1 Adaptive Multi-Hash Divide. BEAM reconstructs the divide groups at the beginning of each iteration from the current active supernodes rather than treating the groups as persistent objects. This is necessary because accepted merges change both the active supernodes and their structural neighborhoods. Consequently, a grouping that was suitable for an

earlier partition need not define an appropriate search space for the current partition.

For iteration t , BEAM computes locality-sensitive min-hash ranks for the active supernodes over multiple hash dimensions [6, 7]. It first orders active supernodes according to one hash dimension. An oversized bucket is recursively refined using additional dimensions until it satisfies the configured maximum group size. If all configured dimensions are exhausted while a bucket remains oversized, BEAM applies a fixed deterministic split.

The divide stage therefore provides two properties needed by the subsequent algorithm. First, it bounds the size of each local candidate-search domain. Second, it assigns each active supernode to exactly one divide group in the current iteration, so endpoint conflicts can be resolved independently within groups.

Divide groups should not be confused with group blocks. A divide group defines candidate eligibility and group-local matching. A group block is a scheduling unit: it combines multiple disjoint groups that are evaluated against one frozen partition state and whose selected merges are committed together.

3.2.2 Sparse Profiles and EA-Proxy Retention. For each active supernode X , BEAM constructs a sparse profile W_X over the current partition. For every current supernode Z ,

$$W_X[Z] = e(X, Z),$$

and coordinates with zero edge count are omitted. Thus, W_X represents the current quotient neighborhood of X . Section ?? describes how these exact counts are obtained and maintained; the candidate policy itself depends only on their values.

BEAM constructs target buckets from the nonzero coordinates of these profiles. Two source supernodes become a proxy pair only if they co-occur in at least one target bucket. For active supernodes X and Y , define their shared-target scope as

$$B(X, Y) = \{Z : W_X[Z] > 0 \text{ and } W_Y[Z] > 0\}.$$

For each $Z \in B(X, Y)$, the proxy uses the pair-cost contribution

$$\Delta_Z(X, Y) = L(X, Z) + L(Y, Z) - L(X \cup Y, Z).$$

The implementation accumulates only positive shared-target contributions and adds a lightweight direct-edge signal

$$h_{\text{dir}}(X, Y) = e(X, Y).$$

The resulting proxy score is

$$\text{proxy_gain}(X, Y) = \sum_{Z \in B(X, Y)} \max\{0, \Delta_Z(X, Y)\} + h_{\text{dir}}(X, Y). \quad (15)$$

Because unnormalized local costs tend to be larger for larger supernodes, BEAM ranks proxy pairs using

$$\text{proxy_ratio}(X, Y) = \frac{\text{proxy_gain}(X, Y)}{\tilde{C}_{\text{old}}(X) + \tilde{C}_{\text{old}}(Y) + \varepsilon}, \quad (16)$$

where $\tilde{C}_{\text{old}}(X)$ and $\tilde{C}_{\text{old}}(Y)$ are profile-derived approximate old-cost normalizers, and $\varepsilon > 0$ prevents division by zero. A fixed endpoint-based tie-breaking rule makes candidate ordering deterministic.

For group Q , let $\text{top}_k(X, Q)$ contain at most k supernodes in $Q \setminus \{X\}$ with the highest proxy rank against X . BEAM forms

$$C_Q = \{ \{X, Y\} : X, Y \in Q, X \neq Y, \\ Y \in \text{top}_k(X, Q) \text{ or } X \in \text{top}_k(Y, Q) \}. \quad (17)$$

The set is deduplicated because the same unordered pair may be selected by both endpoints or generated through multiple shared targets.

Equations (15)–(17) define a budget-allocation mechanism, not an approximation guarantee for the final merge decision. The shared-target scope may omit terms from the exact affected neighborhood, positive-only accumulation may change pair ordering, and the direct-edge term is heuristic. BEAM therefore neither accepts nor rejects a merge based on the proxy score. Every pair in C_Q is subsequently evaluated under the exact correction-set objective.

3.3 Exact Encoding-Aware Gain Evaluation

For every retained pair $\{X, Y\} \in C_Q$, BEAM computes the exact gain defined in Equation (11) against the frozen block state $\mathcal{S}_B$. The full local-to-global equivalence was established in Section 2.3; here we specify how the required quantities are obtained from the current sparse profiles.

The affected external neighborhood is

$$N_{XY} = (\text{supp}(W_X) \cup \text{supp}(W_Y)) \setminus \{X, Y\}.$$

For each $Z \in N_{XY}$, the merged edge count is

$$e(M_{XY}, Z) = e(X, Z) + e(Y, Z).$$

The merged supernode size and internal edge count are

$$|M_{XY}| = |X| + |Y|,$$

and

$$e(M_{XY}, M_{XY}) = e(X, X) + e(Y, Y) + e(X, Y).$$

Together with the exact supernode sizes, these statistics instantiate the capacities and sparse/dense pair costs in Equations (1)–(4). BEAM then evaluates $L_{\text{before}}(X, Y)$ and $L_{\text{after}}(X, Y)$ using Equations (9)–(10).

Algorithm 2 presents this step as an abstract retained-edge evaluation routine. It deliberately leaves the storage representation and execution backend unspecified. Those choices affect how the required statistics are accessed, but not the edge set, edge weights, or positive-gain criterion.

PROPOSITION 3.1 (EXACTNESS OF RETAINED-EDGE WEIGHTS).
For every retained pair $\{X, Y\} \in C_Q$, the value computed in Line 5 of Algorithm 2 equals

$$\text{Cost}(\mathcal{S}_B) - \text{Cost}(\mathcal{S}_B \downarrow (X, Y)).$$

Algorithm 3: Exact Retained-Candidate Graph Construction

Input: retained set C_Q ; profiles $\{W_X : X \in Q\}$; frozen partition state $\mathcal{S}_B$.

Output: weighted positive-gain graph $H_Q = (Q, C_Q^+, w)$.

```

1:  $C_Q^+ \leftarrow \emptyset$ ;  $w \leftarrow \emptyset$ .
2: for each retained pair  $\{X, Y\} \in C_Q$  do
3:   Form  $N_{XY} \leftarrow (\text{supp}(W_X) \cup \text{supp}(W_Y)) \setminus \{X, Y\}$ .
4:   Compute  $|M_{XY}|$ ,  $e(M_{XY}, M_{XY})$ , and  $e(M_{XY}, Z)$  for  $Z \in N_{XY}$ .
5:    $g \leftarrow L_{\text{before}}(X, Y) - L_{\text{after}}(X, Y)$ .
6:   if  $g > 0$  then
7:      $C_Q^+ \leftarrow C_Q^+ \cup \{\{X, Y\}\}$ .
8:      $w(X, Y) \leftarrow g$ .
9:   end if
10: end for
11: return  $H_Q = (Q, C_Q^+, w)$ .
```

Figure 2: Construction of the exact positive-gain candidate graph. Each retained pair is evaluated against the same frozen block state. The routine is backend-neutral and may be executed in batches.

Thus, every edge weight in H_Q is the exact change in the global partition objective caused by applying that merge in isolation.

PROOF. The profiles provide the exact incident counts $e(X, Z)$ and $e(Y, Z)$ for every Z in the union of their supports. Together with $|X|$, $|Y|$, $e(X, X)$, $e(Y, Y)$, and $e(X, Y)$, these counts determine all terms in $L_{\text{before}}(X, Y)$ and $L_{\text{after}}(X, Y)$. All partition-cost terms outside this affected scope are unchanged. The claim therefore follows directly from Equation (12). $\square$

Proposition 3.1 concerns a single edge evaluated against the frozen state $\mathcal{S}_B$. It does not imply that the isolated gains of several selected edges are additive after the entire block is committed. BEAM uses these exact isolated gains as objective-faithful edge weights for block-local selection.

3.4 Greedy Positive-Gain Matching

Algorithm 3 shows the complete group-local selection procedure. Profile-based proposal first produces C_Q . Algorithm 2 then constructs the weighted positive-gain graph

$$H_Q = (Q, C_Q^+, w)$$

defined in Section 2.4. The matching step operates directly on this graph; proxy scores no longer participate in merge selection.

BEAM orders the edges of C_Q^+ by non-increasing exact gain, using a fixed canonical endpoint order to break ties. It accepts an edge if neither endpoint has been used by an earlier accepted edge. The result $\mathcal{M}_Q$ is therefore endpoint-disjoint.

Because divide groups are disjoint within one iteration, no supernode appears in two group-local matching problems in

Algorithm 2: Greedy Gain Matching within One Group

Input: divide group Q ; candidate budget k ; frozen partition state $\mathcal{S}_B$.

Output: conflict-free merge set $\mathcal{M}_Q$.

```

1: Construct the exact sparse profiles  $\{W_X : X \in Q\}$  under  $\mathcal{S}_B$ .
2:  $C_Q \leftarrow \text{PROFILECANDIDATESPARSIFICATION}(Q, \{W_X\}, k)$ .
3:  $H_Q = (Q, C_Q^+, w) \leftarrow \text{EXACTRETAINEDCANDIDATEGRAPH}(C_Q, \{W_X\}, \mathcal{S}_B)$ .
4: Order the edges of  $C_Q^+$  by non-increasing  $w$ , using fixed deterministic tie breaking.
5:  $\mathcal{M}_Q \leftarrow \emptyset$ ;  $\text{used} \leftarrow \emptyset$ .
6: for each edge  $\{X, Y\} \in C_Q^+$  in this order do
7:   if  $X \notin \text{used}$  and  $Y \notin \text{used}$  then
8:      $\mathcal{M}_Q \leftarrow \mathcal{M}_Q \cup \{\{X, Y\}\}$ .
9:      $\text{used} \leftarrow \text{used} \cup \{X, Y\}$ .
10:  end if
11: end for
12: return  $\mathcal{M}_Q$ .

```

Figure 3: Group-local proposal, exact validation, and conflict-free selection. Matching uses only the exact positive-gain graph H_Q ; the proxy scores used to form C_Q do not enter the matching rule.

the same block. Consequently,

$$\mathcal{M}_B = \bigcup_{Q \in B} \mathcal{M}_Q$$

is also endpoint-disjoint. This establishes structural compatibility of the selected merges, but does not assert that their objective effects are mutually independent.

PROPOSITION 3.2 (RETAINED-GRAPH MATCHING GUARANTEE). *Fix a divide group Q and its retained positive-gain candidate graph $H_Q = (Q, C_Q^+, w)$. Let $\mathcal{M}_Q^g$ be the matching returned by Algorithm 3, and let $\mathcal{M}_Q^*$ be a maximum-weight matching on the same graph H_Q . Then*

$$\sum_{e \in \mathcal{M}_Q^g} w(e) \geq \frac{1}{2} \sum_{e \in \mathcal{M}_Q^*} w(e).$$

PROOF. Consider the edges selected by the greedy algorithm in selection order. When a greedy edge $e = \{X, Y\}$ is selected, all unprocessed edges incident to X or Y become unavailable. Since $\mathcal{M}_Q^*$ is a matching, at most two of its edges can be incident to the endpoints of e : at most one through X and at most one through Y .

The greedy algorithm processes edges in non-increasing weight order. Therefore, every optimal edge first blocked by e has weight at most $w(e)$. Charge each edge of $\mathcal{M}_Q^*$ either to itself when it is also selected by greedy, or to the first greedy edge that blocks it. Each greedy edge receives charge from at most two optimal edges, and its total charge is therefore at most $2w(e)$. Summing over the greedy matching gives

$$w(\mathcal{M}_Q^g) \leq 2w(\mathcal{M}_Q^*),$$

which proves the result. $\square$

The scope of Proposition 3.2 is intentionally limited. It bounds only the loss introduced by greedy conflict resolution after profile-based sparsification and exact positive-gain filtering have fixed H_Q . It does not compare C_Q^+ with the complete graph of all possible supernode pairs and does not provide an approximation guarantee for the globally optimal final partition.

3.5 Complexity and Approximation Scope

Consider one divide group Q . For each profile coordinate Z , let

$$B_Z = \{X \in Q : W_X[Z] > 0\}$$

be the corresponding target bucket. Enumerating source pairs that co-occur in these buckets requires

$$O\left(\sum_Z |B_Z|^2\right)$$

pair-generation work before aggregation and top- k retention. This bound reflects the sparse bucket-induced construction; its worst case is $O(|Q|^2)$ when one target bucket contains the entire group.

Let d_X be the number of accumulated proxy partners for source supernode X . Deterministic full ordering of these partners costs

$$O\left(\sum_{X \in Q} d_X \log d_X\right),$$

after which at most k partners are retained per source. The number of directed top- k selections is therefore at most $|Q|k$, and the number $c = |C_Q|$ of unordered retained pairs after deduplication satisfies

$$c \leq |Q|k.$$

Exact evaluation of the retained set requires

$$O\left(\sum_{\{X, Y\} \in C_Q} |\text{supp}(W_X) \cup \text{supp}(W_Y)|\right)$$

local-profile work. Let $c_+ = |C_Q^+|$. Ordering the exact-positive edges costs $O(c_+ \log c_+)$, and greedy matching costs $O(c_+)$ using a marker set for used endpoints.

These bounds expose two distinct approximations. First, profile-based top- k retention approximates the current merge search space by constructing C_Q . Second, descending-gain matching approximates maximum-weight matching on the fixed graph H_Q . In contrast, the weight assigned to every retained edge is exact under the current partition.

The overall summarization procedure remains heuristic with respect to the globally optimal flat summary. The candidate graph is sparse, divide groups restrict pair eligibility, candidate graphs and gains are regenerated as the partition evolves, and isolated edge gains may interact when a block is committed. Proposition 3.2 therefore cannot be composed across groups, blocks, or iterations into a global approximation ratio.

3.6 Blocked Merge Application

For a group block B , BEAM combines the disjoint group-local matchings into

$$\mathcal{M}_B = \bigcup_{Q \in B} \mathcal{M}_Q.$$

All pairs in $\mathcal{M}_B$ were proposed, evaluated, and selected against the same frozen state $\mathcal{S}_B$. BEAM commits them only after selection has completed for every group in the block.

For each selected pair $\{X, Y\} \in \mathcal{M}_B$, BEAM replaces X and Y by

$$M_{XY} = X \cup Y.$$

For every current supernode Z outside the pair, the updated incident edge count is

$$e(M_{XY}, Z) = e(X, Z) + e(Y, Z),$$

and the internal edge count of the new supernode is

$$e(M_{XY}, M_{XY}) = e(X, X) + e(Y, Y) + e(X, Y).$$

These identities define the partition mutation and the quotient statistics visible to later blocks. They are the same combinatorial identities used during exact gain evaluation, but here they update the actual summary state rather than a hypothetical merged state.

Endpoint-disjointness ensures that every original supernode has a unique image in the partition produced by the block. It does not imply that the isolated gains of selected pairs are additive. In particular, pair costs between supernodes belonging to two different selected merges can change when both merged supernodes are created. The blocked policy should therefore be understood as a deterministic commit semantics: it freezes proposal and decision within the block and makes the resulting partition visible to subsequent blocks.

After a block is committed, any profile, candidate edge, or gain whose definition depends on the changed partition is considered stale. BEAM refreshes the required summary statistics before they are used for a later block or iteration. The next section describes how this state can be maintained persistently without changing the candidate policy, exact-gain definition, matching order, or block-commit semantics.

After the final iteration, BEAM encodes the resulting partition into the standard flat correction-set representation (P, C^+, C^-) .

4 BATCHED GAIN EVALUATION

Section 3 defined the exact local objective. This section makes explicit how BEAM turns that objective into a batched scoring workload. After deduplication, the retained candidates from one or more prepared groups are packed into a gain-evaluation unit

$$U = (\text{cand_src}, \text{cand_dst}, \text{prof_ptr}, \text{prof_nbr}, \\ \text{prof_val}, \text{size}, \text{self}),$$

where $\text{cand_src}[i]$ and $\text{cand_dst}[i]$ identify candidate i . The array prof_ptr gives the slice boundaries of each sparse profile, prof_nbr stores the supernode identifiers of nonzero profile entries, prof_val stores the corresponding edge counts, $\text{size}[X] = |X|$, and $\text{self}[X] = e(X, X)$. If several groups are packed into one batch, the host additionally stores group or batch offsets so that the returned gains can be mapped back to the corresponding group-local candidate graphs for matching.

The key point is that grouping and scoring play different roles. Grouping is a host-side search-space restriction: it determines which supernodes may become candidate partners and which candidate scores will later compete in the same greedy matching step. The batched evaluator only scores retained candidate pairs under the current partition; merge application and conflict resolution remain part of the higher-level host-side algorithm.

Algorithm 3 describes the per-candidate computation. For candidate $i = (X, Y)$, one worker reads the sparse profile slices of W_X and W_Y , traverses $\text{supp}(W_X) \cup \text{supp}(W_Y)$, and treats missing entries as zero. This directly yields the external counts $e(X, Z)$, $e(Y, Z)$, and $e(M, Z) = e(X, Z) + e(Y, Z)$ needed for every affected supernode Z . The same worker also computes the merged self statistics $|M| = |X| + |Y|$ and $e(M, M) = e(X, X) + e(Y, Y) + e(X, Y)$, evaluates L_{before} and L_{after} , and writes one exact gain value to $G[i]$. The work is therefore parallel across candidates but still exact with respect to the local correction-set objective.

Among the stages in the merge loop, gain evaluation is the one that admits the clearest batch structure, while candidate sparsification and summary updates remain tied to irregular control flow and state mutation. Batched evaluation therefore isolates the part of the algorithm most amenable to regular parallel execution. In the current implementation, the same packed representation can be evaluated by CPU threads or transferred to CUDA for device-side parallel scoring, whereas candidate generation, greedy matching, merge application, and final correction-set encoding remain in the host-side flat pipeline.

The benefit of the current CUDA instantiation is workload dependent. If the batched candidate set is large enough and per-candidate gain evaluation is sufficiently expensive, device execution may amortize transfer and launch overhead. If candidate sets are small or the local objective is cheap, the same overheads may limit or eliminate the benefit. The paper therefore characterizes the CUDA implementation empirically across workloads.

4.1 CUDA Mapping

The current CUDA backend treats retained candidates as independent scoring tasks rather than attempting to keep the full summary state on the device. The key mapping is that candidate scoring is parallelized across candidate pairs: each device worker receives one retained candidate together with offsets into the packed sparse-profile arrays, computes the corresponding local gain, and writes one output value.

Because candidate profile lengths vary, different workers may traverse different numbers of sparse entries, so the CUDA workload is irregular even though the interface is batched.

To make this workload executable on the GPU, the host first prepares one or more groups, aggregates their sparse profiles at the current supernode level, and flattens the resulting data into contiguous arrays. Host-to-device transfer therefore moves candidate endpoints, profile offsets, profile neighbor identifiers, profile edge-count values, supernode sizes, and self-loop counts. Device-to-host transfer returns only the scored gain array. Figure 7 and Table 6 report these three component counters separately as H2D, kernel, and D2H time.

The blocked schedule also explains why the number of scoring slices and kernel launches can be large. The host repeatedly forms group blocks and invokes the CUDA scorer on those blocked workloads. If a blocked workload is still too large, the implementation may further split it into candidate-budgeted kernel chunks. The reported **Block Size** in Table 6 refers to the group-block size used by the blocked schedule, while the large **Scoring Slices** and **Kernel Launches** counts reflect repeated blocked or sub-batched scoring invocations across all iterations.

5 EXPERIMENTAL EVALUATION

5.1 Experimental Questions

Our evaluation focuses on BEAM’s merge-stage organization within the flat correction-set model and separately characterizes the CUDA backend used for retained gain evaluation. We therefore organize the evaluation around three core questions and one implementation analysis question.

- **RQ1.** Does BEAM improve the speed–quality trade-off over SWeG, the same-model flat correction-set baseline?
- **RQ2.** What tradeoff does BEAM offer relative to SLUGGER, a more expressive hierarchical summarization baseline?
- **RQ3.** Which stages dominate BEAM’s runtime, and how much of the execution is concentrated in candidate sparsification and gain evaluation?
- **RQ4.** As an implementation analysis, what role does the CUDA backend play in the end-to-end BEAM implementation, and what costs arise from batched device-side gain evaluation?

These questions separate same-model comparison, cross-model context, and implementation-level analysis. This distinction matters because BEAM and SWeG produce flat correction-set summaries, whereas SLUGGER uses a hierarchical representation, while RQ4 focuses only on one scoring backend inside BEAM.

5.2 Datasets and Baselines

We evaluate BEAM on eleven graph datasets with different sizes, sparsity levels, and graph types. Table 2 summarizes the datasets used in the evaluation. The collection includes

Table 2: Datasets used in the evaluation.

Dataset	Nodes	Edges	Type
Facebook(FA)	4,039	88,234	Social
Email-Enron(EM)	36,692	183,831	Email
Amazon(AM)	334,863	925,872	Network
DBLP(DB)	317,080	1,049,866	Collaboration
Youtube(YO)	1,134,890	2,987,624	Social
RoadNet(RN)	1,965,206	2,766,607	Network
Livejournal(LJ)	3,997,962	34,681,189	Social
Hollywood(HO)	1,107,243	56,375,711	Collaboration
Orkut(OR)	3,072,441	117,185,083	Network
UK-2002(U2)	18,520,343	261,787,258	Hyperlinks
UK-2005(U5)	39,459,923	783,027,125	Hyperlinks

social, email, collaboration, product, road, and hyperlink graphs. Most datasets are from SNAP [15], while the largest web graphs are from the LAW/WebGraph ecosystem [5]. The overall comparison uses the ten datasets for which all three methods have corresponding results. Orkut is excluded from the overall comparison because matching SWeG and SLUGGER results are unavailable, but it is included in BEAM’s runtime-breakdown and CUDA-backend analysis.

We use the SWeG implementation released in the LDME project and the original SLUGGER implementation released by its authors. Both baselines are CPU implementations. All methods are evaluated on the same machine and with the same CPU thread setting. BEAM uses the CPU for grouping, candidate generation, greedy matching, merge application, and final encoding, and uses the NVIDIA Tesla P100 as one backend for batched gain evaluation. We report this as an end-to-end system comparison under a common hardware setting, and separately isolate the CUDA scoring cost in Section 5.7.

We compare BEAM with two baselines. **SWeG** is the primary same-model baseline because it is a scalable flat correction-set summarization method and was shown to dominate earlier lossless summarization methods in speed–quality tradeoff [2, 11, 19–21]. We start from the single-thread SWeG implementation released in the LDME project and reproduce a multithreaded version for our runs. **SLUGGER** is included as a hierarchical contrast baseline. Since SLUGGER uses a different summary representation, this comparison should be interpreted as cross-model context rather than a same-semantics comparison. For SLUGGER, we use the original code released with the paper.

All methods are evaluated under the same hardware and software environment. The experiments run on a dual-socket server with two Intel Xeon Silver 4316 CPUs at 2.30 GHz, providing 40 physical cores and 80 hardware threads in total, and an NVIDIA Tesla P100 GPU. Unless otherwise stated, all methods are run for 20 iterations. The reported BEAM-CUDA runs use top- $k = 16$, a group-block size of 64, and a candidate batch budget of 65,536. In this section, “BEAM” refers to the implementation using batched CUDA gain evaluation.

5.3 Metrics

We report two end-to-end metrics: relative output size and total runtime. Relative output size measures the size of the compressed representation normalized by the number of original graph edges:

$$\text{RelSize} = \frac{|P| + |C^+| + |C^-|}{|E|},$$

where P is the superedge set and C^+ and C^- are the positive and negative correction sets. Lower relative size indicates better compression. Runtime is the total wall-clock execution time, where lower is better.

For BEAM, we further report stage-level runtime. We separate divide time, merge time, encode time, candidate-generation time, gain-scoring time, merge share, and scoring share. Here merge share and scoring share denote the fractions of total runtime spent in merge-related work and in the combined prepare-and-exact-scoring portion of the merge stage, respectively. For the blocked implementation, **Prepare Wall** denotes the CPU-side wall-clock time spent preparing retained candidates before exact scoring, including sparse-profile refresh, target-bucket construction, EA-proxy accumulation, top- k retention, candidate deduplication, and construction of the retained candidate data consumed by the scoring backend. **Gain Scoring Wall** denotes only the wall-clock time of the exact-gain evaluator: the CPU exact-gain evaluator time for the CPU backend and the CUDA scoring wall time for the CUDA backend. For CUDA, we also report H2D, kernel, D2H, scoring slices, and kernel launches as component counters rather than as a strictly additive decomposition of CUDA wall-clock time.

5.4 RQ1: Same-Model Comparison with SWeG

Table 3, Figure 4, and Figure 5 compare BEAM with SWeG on the ten common datasets. This is the primary comparison because both methods use the flat correction-set representation.

In this end-to-end implementation comparison, BEAM is faster than SWeG on all ten common datasets. The geometric mean speedup over SWeG is $2.69\times$. In terms of compression, BEAM produces smaller relative output size on all ten common datasets. These results show that BEAM improves the speed-quality tradeoff within the same summary model: its candidate-sparse merge-stage organization reduces runtime while its exact local gain scoring preserves or improves compression quality across the common datasets.

BEAM relies on adaptive divide grouping and top- k profile-based candidate retention to restrict the candidate graph. This design reduces search cost while still preserving the same-model speed-quality tradeoff against SWeG under the reported settings.

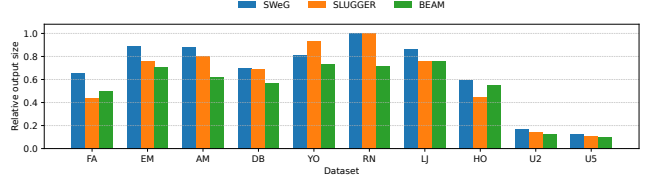


Figure 4: RQ1–RQ2: Relative output size on common datasets. Relative size is the compressed representation size divided by the number of original edges; lower is better. BEAM improves over SWeG on all ten common datasets, while SLUGGER remains a hierarchical contrast with mixed compression tradeoffs.

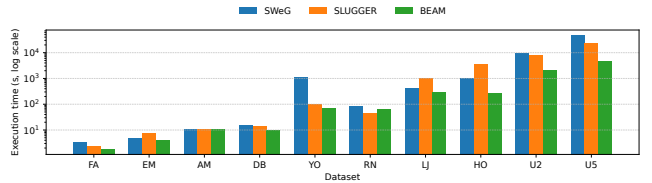


Figure 5: RQ1–RQ2: Execution time on common datasets under 20 iterations. The y-axis uses a logarithmic scale; lower is better. BEAM is faster than SWeG on all ten common datasets and faster than SLUGGER on nine of them.

5.5 RQ2: Cross-Model Comparison with SLUGGER

We also compare BEAM with SLUGGER to provide cross-model context against a more expressive hierarchical summarization method. BEAM outputs a flat correction-set summary, whereas SLUGGER uses a hierarchical representation.

Under the same experimental setting, BEAM is faster than SLUGGER on nine of the ten common datasets; the exception is RoadNet. The geometric mean speedup over SLUGGER is $2.29\times$. Compression quality remains mixed. BEAM obtains smaller relative size on EM, AM, DB, YO, RN, LJ, U2, and U5, while SLUGGER obtains smaller relative size on FA and HO. This result is consistent with the difference between the two representations: SLUGGER’s hierarchy can express some structures more compactly, while BEAM keeps the flat representation and focuses on repeated merge-stage organization. The comparison is therefore best read as cross-model context, showing that a flat correction-set method can remain competitive in compression while offering substantially lower runtime in this setting.

5.6 RQ3: Runtime Concentration in the Merge Stage

Figure 6 and Table 4 show that merge-related work accounts for a large fraction of BEAM’s runtime and dominates on most datasets in the batch-ea-blocked CUDA configuration.

Table 3: RQ1–RQ2: Overall comparison on the ten common datasets. Lower relative size and lower runtime are better.

Dataset	SWeG Rel. Size	SWeG Time (s)	SLUGGER Rel. Size	SLUGGER Time (s)	BEAM Rel. Size	BEAM Time (s)	Speedup vs SWeG	Speedup vs SLUGGER
FA	0.657	3.27	0.442	2.31	0.497	1.87	1.75×	1.24×
EM	0.891	5.03	0.763	7.63	0.711	3.99	1.26×	1.91×
AM	0.879	10.55	0.802	11.11	0.619	10.35	1.02×	1.07×
DB	0.698	15.34	0.690	13.82	0.571	10.11	1.52×	1.37×
YO	0.815	1127.86	0.935	103.32	0.735	68.90	16.37×	1.50×
RN	0.999	87.65	1.000	46.89	0.716	62.36	1.41×	0.75×
LJ	0.865	435.19	0.764	1061.45	0.756	299.03	1.46×	3.55×
HO	0.598	987.03	0.445	3525.73	0.549	263.78	3.74×	13.37×
U2	0.173	9442.87	0.144	8071.43	0.126	2024.26	4.66×	3.99×
U5	0.130	45681.93	0.110	24170.22	0.101	4585.81	9.96×	5.27×
Geo. mean speedup	—	—	—	—	—	—	2.69×	2.29×

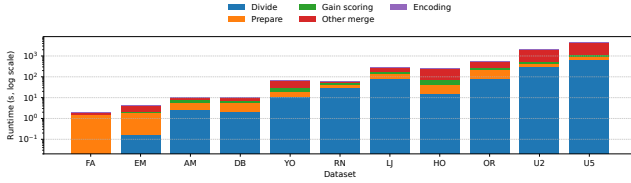


Figure 6: RQ3: Runtime breakdown of BEAM. The stacked bars separate divide, prepare, gain scoring, other merge work, and final encoding. Merge-related stages dominate total runtime on most datasets.

Merge share ranges from 50.0% to 97.9% of total runtime. The combined prepare-and-gain-scoring share ranges from 21.3% to 79.4%, while gain scoring alone ranges from 2.0% to 19.0%. Prepare work is often the largest visible component, while exact gain scoring remains a persistent but smaller part of merge cost across datasets. This pattern supports the merge-stage bottleneck claim: the main scalability challenge in BEAM is not final correction-set encoding, but the repeated construction, scoring, selection, and application of candidate merges under a changing partition.

At the same time, the breakdown shows that gain evaluation alone is not the only bottleneck. Prepare work, including sparse-profile refresh, target-bucket construction, EA-proxy accumulation, top- k retention, deduplication, and retained-candidate packing, can dominate on several datasets. This observation explains why GPU acceleration of gain scoring improves only one component of the pipeline and why the surrounding blocked merge machinery remains important for end-to-end performance.

5.7 RQ4: CUDA Gain-Evaluation Cost

To separate the benefit of BEAM’s merge-stage organization from the benefit of the CUDA scoring backend, we compare two implementations of the same batch-ea-blocked algorithm: one using CPU gain scoring and one using CUDA gain scoring. Both configurations use the same divide procedure, retained candidate generation, greedy positive-gain matching, deferred merge updates, and final correction-set

encoding. The two backends produce identical summary outputs in our runs, including cost ratio, encoding cost, supernode count, and the final correction-set edge counts; only the execution backend for retained gain evaluation changes.

Table 5 shows that CUDA improves end-to-end runtime on 9 of the 11 datasets, with a geometric mean end-to-end speedup of 1.19×. The benefit is workload dependent. On small datasets such as FA and EM, CUDA overhead outweighs device-side scoring gains, leading to slowdowns. On larger or more scoring-intensive datasets such as AM, DB, RN, and LJ, CUDA provides clearer end-to-end acceleration. CUDA therefore acts as a backend for retained-candidate scoring when that workload is large enough, while end-to-end gains still depend mainly on BEAM’s candidate-sparse merge organization.

Figure 7 and Table 6 further characterize where time is spent inside the CUDA scoring path. We report CUDA Total as the wall-clock time of the high-level CUDA scoring call, and we separately report H2D, kernel, D2H, scoring slices, and kernel launches as component counters. Because CUDA streams may overlap transfer and kernel execution, these component counters are diagnostic measurements and may not sum to CUDA Total.

The cost balance varies across datasets. On the largest common workload, U5, kernel execution takes 136.373s and host-to-device transfer adds 19.298s. U5 also issues 1,618,975 scoring slices and 1,607,709 kernel launches, reflecting the blocked execution schedule rather than a single monolithic GPU run. BEAM repeatedly forms group blocks and invokes the CUDA backend on these blocked workloads; large blocked workloads may be further split by the candidate batch budget. Thus, the reported slice and launch counts should be interpreted as repeated blocked or sub-batched gain-scoring invocations.

Across datasets, CUDA gain-evaluation time accounts for 2.0%–19.0% of end-to-end BEAM runtime. Together with Table 5, this shows that CUDA accelerates retained gain scoring when the workload is large enough, while the end-to-end performance also depends on CPU-side grouping, candidate generation, prepare work, greedy matching, merge application, and final encoding. These results position CUDA as a scoring backend within BEAM’s candidate-sparse, encoding-aware merge-stage organization.

Table 4: RQ3: BEAM runtime breakdown across datasets.

Dataset	Total (s)	Divide (s)	Merge (s)	Encode (s)	Prepare (s)	Gain Scoring (s)	Merge Share	Prep.+Gain Share	Gain Share	Candidates
FA	1.87	0.02	1.83	0.02	0.36	0.04	97.9%	21.3%	2.0%	11,713
EM	3.99	0.17	3.79	0.04	1.82	0.32	94.8%	53.7%	8.0%	359,890
AM	10.35	2.65	7.48	0.22	2.32	1.82	72.3%	40.1%	17.6%	299,875
DB	10.11	2.18	7.68	0.24	2.47	1.92	76.0%	43.5%	19.0%	380,762
YO	68.90	11.33	56.84	0.71	38.33	10.56	82.5%	70.9%	15.3%	13,548,785
RN	62.36	30.24	31.20	0.88	8.55	11.53	50.0%	32.2%	18.5%	862,286
LJ	299.03	81.19	208.04	9.73	112.69	44.28	69.6%	52.5%	14.8%	4,175,972
HO	263.78	15.69	236.21	11.87	183.70	25.65	89.5%	79.4%	9.7%	7,659,141
OR	581.63	82.22	467.60	31.77	273.60	69.38	80.4%	59.0%	11.9%	1,801,651
U2	2024.26	303.31	1679.29	41.29	1465.70	99.32	83.0%	77.3%	4.9%	310,296,688
U5	4585.81	651.09	3789.20	144.87	3280.82	231.95	82.6%	76.6%	5.1%	823,853,836

Table 5: End-to-end comparison between CPU and CUDA exact-gain scoring backends under the same BEAM batch-ea-blocked pipeline. Both configurations use the same adaptive divide procedure, EA-proxy candidate retention, greedy matching, merge update, and final encoding; only the retained exact-gain scoring backend changes. CUDA scoring wall share is computed as `cuda_total_ms / runtime_end_to_end_ms`.

Dataset	CPU-backend E2E Time (s)	CUDA-backend E2E Time (s)	E2E Speedup	CUDA Scoring Wall Share
FA	0.58	1.87	0.31×	2.0%
EM	3.42	3.99	0.86×	8.0%
AM	20.38	10.35	1.97×	17.6%
DB	19.09	10.11	1.89×	19.0%
YO	90.44	68.90	1.31×	15.3%
RN	163.27	62.36	2.62×	18.5%
LJ	397.39	299.03	1.33×	14.8%
HO	276.52	263.78	1.05×	9.7%
OR	663.50	581.63	1.14×	11.9%
U2	2254.13	2024.26	1.11×	4.9%
U5	4993.09	4585.81	1.09×	5.1%

Table 6: RQ4: CUDA scoring-path measurements for BEAM. CUDA Total is the wall-clock time of the high-level CUDA scoring path. H2D, kernel, and D2H are component counters and may not sum to CUDA Total because CUDA streams can overlap.

Dataset	Candidates	Block Size	CUDA Total (s)	H2D (s)	Kernel (s)	D2H (s)	Scoring Slices	Kernel Launches	Total Time (s)	CUDA Share
FA	11,713	64	0.037	0.004	0.020	0.002	168	168	1.87	2.0%
EM	359,890	64	0.319	0.022	0.188	0.027	2,774	2,774	3.99	8.0%
AM	299,875	64	1.822	0.162	0.670	0.267	28,160	28,148	10.35	17.6%
DB	380,762	64	1.922	0.157	0.842	0.248	26,031	26,025	10.11	19.0%
YO	13,548,785	64	10.550	0.652	5.814	1.106	116,126	115,821	68.90	15.3%
RN	862,286	64	11.512	1.139	2.606	2.142	229,609	229,476	62.36	18.5%
LJ	4,175,972	64	44.245	3.231	27.546	3.302	339,114	336,759	299.03	14.8%
HO	7,659,141	64	25.648	2.256	18.694	0.449	40,442	40,442	263.78	9.7%
OR	1,801,651	64	69.360	7.302	46.504	1.658	138,616	136,932	581.63	11.9%
U2	310,296,688	64	99.230	8.474	52.065	9.485	928,803	922,330	2024.26	4.9%
U5	823,853,836	64	231.791	19.298	136.373	17.071	1,618,975	1,607,709	4585.81	5.1%

5.8 Parameter Sensitivity

We evaluate BEAM’s sensitivity to its main pruning parameter, $\text{top-}k$, on FA, EM, AM, DB, and YO using the same batch-ea-blocked CUDA configuration as in the main experiments. Figure 8 reports geometric-mean runtime, relative output size, and retained candidate count normalized to the default $k = 16$. Smaller k reduces retained candidates but degrades compression, while increasing k from 16 to 32 gives

only a small relative-size improvement at the cost of many more retained candidates. We therefore use $k = 16$ as a fixed default.

We also tested group-block sizes in $\{32, 64, 128, 256\}$ and CUDA candidate-batch budgets from 4,096 to 65,536. Relative output size was essentially unchanged in these sweeps, and runtime differences mainly reflected batching and launch

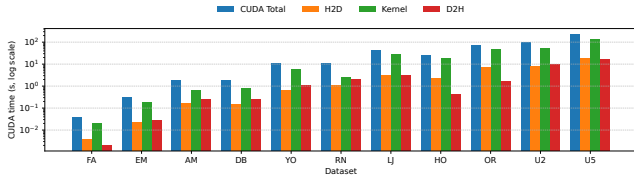


Figure 7: RQ4: CUDA scoring-path counters for BEAM’s batched gain evaluation. The grouped bars report CUDA Total wall time together with H2D, kernel, and D2H component counters. The component counters are not expected to sum to CUDA Total when stream overlap is enabled.

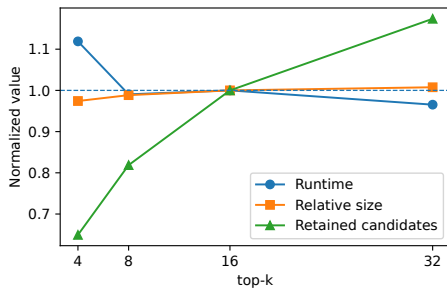


Figure 8: Sensitivity to top- k on five datasets. Values are geometric means normalized to the default $k = 16$. Increasing k from 16 to 32 slightly improves relative output size but retains more candidates, so BEAM uses $k = 16$ as a fixed stable default.

overhead. We use block size 64 and candidate budget 65,536 as stable defaults rather than tuning them per dataset.

6 RELATED WORK

Graph summarization is closely related to graph compression and compact graph representation. General graph-compression methods reduce storage through vertex ordering, reference encoding, interval encoding, locality-aware layouts, recursive partitioning, and compressed graph processing [1, 3–5, 8, 9, 16, 23]. These methods usually encode a fixed graph compactly. In contrast, graph summarization constructs a smaller structural representation, such as supernodes, superedges, and correction information, that can reconstruct or approximate the original graph [17, 19].

BEAM is closest to flat correction-set graph summarization. Prior flat methods, including SWeG, use divide-and-merge pipelines to restrict candidate search and construct compact summaries under flat correction-set semantics [2, 12, 13, 21]. BEAM keeps the same representation family but changes the merge-stage organization: lightweight profiles are used only to sparsify candidate search, while retained candidates are scored by exact local correction-set gain. Hierarchical summarizers such as SLUGGER use nested supernodes and therefore represent a different model class [14,

25]; we use them as cross-model baselines rather than same-model competitors.

Scalable and parallel graph-processing systems optimize traversal, filtering, sparse linear algebra, and frontier-style computation over relatively fixed graph representations [18, 22–24, 26]. BEAM addresses a different bottleneck: in flat graph summarization, the objects being processed are evolving supernodes, candidate pairs, local neighborhoods, and merge decisions. BEAM therefore isolates exact local gain evaluation as a batched scoring step while keeping grouping, matching, and partition updates in the host-side summarization pipeline.

7 CONCLUSION

We presented BEAM, an encoding-aware greedy merging algorithm for scalable flat correction-set graph summarization. BEAM preserves the flat summary model but reorganizes the repeated merge stage by separating cheap candidate sparsification from exact local correction-set gain evaluation. This design enables batched retained-candidate scoring while keeping conflict-free matching and partition updates in the standard flat pipeline. Experiments on ten common datasets show that BEAM is faster than SWeG on all datasets and faster than SLUGGER on nine, with geometric mean speedups of $2.69\times$ and $2.29\times$, respectively. BEAM also improves relative output size over SWeG on all common datasets, while the comparison with hierarchical SLUGGER remains mixed.

ACKNOWLEDGMENTS

This work was supported in part by the National Key R&D Program of China under Grant No. 2023YFB3002702 and in part by the Creative Research Groups Program of the National Natural Science Foundation of China under Grant No. 62321003. Corresponding author: Guoqing Xiao.

REFERENCES

- [1] Alberto Apostolico and Guido Drovandi. 2009. Graph Compression by BFS. *Algorithms* 2, 3 (2009), 1031–1044.
- [2] Maham Anwar Beg, Muhammad Ahmad, Arif Zaman, and Imdadullah Khan. 2018. Scalable Approximation Algorithm for Graph Summarization. In *Advances in Knowledge Discovery and Data Mining - 22nd Pacific-Asia Conference, PAKDD 2018, Melbourne, VIC, Australia, June 3-6, 2018, Proceedings, Part III (Lecture Notes in Computer Science)*, Dinh Q. Phung, Vincent S. Tseng, Geoffrey I. Webb, Bao Ho, Mohadeseh Ganji, and Lida Rashidi (Eds.). Springer, 502–514.
- [3] Maciej Besta and Torsten Hoefer. 2018. Survey and Taxonomy of Lossless Graph Compression and Space-Efficient Graph Representations. *CoRR* abs/1806.01799 (2018).
- [4] Paolo Boldi, Marco Rosa, Massimo Santini, and Sebastiano Vigna. 2011. Layered label propagation: a multiresolution coordinate-free ordering for compressing social networks. In *Proceedings of the 20th International Conference on World Wide Web (Hyderabad, India) (WWW ’11)*. Association for Computing Machinery, New York, NY, USA, 587–596.
- [5] Paolo Boldi and Sebastiano Vigna. 2004. The WebGraph Framework I: Compression Techniques. In *Proceedings of the 13th International Conference on World Wide Web, WWW 2004, New York, NY, USA, May 17-22, 2004*, Stuart I. Feldman, Mike Uretsky, Marc Najork, and Craig E. Wills (Eds.). ACM, 595–602.
- [6] Andrei Z. Broder. 1997. On the resemblance and containment of documents. In *Compression and Complexity of SEQUENCES*

- 1997, Positano, Amalfitan Coast, Salerno, Italy, June 11-13, 1997, *Proceedings*, Bruno Carpentieri, Alfredo De Santis, Ugo Vaccaro, and James A. Storer (Eds.). IEEE, 21–29.
- [7] Andrei Z. Broder, Moses Charikar, Alan M. Frieze, and Michael Mitzenmacher. 2000. Min-Wise Independent Permutations. *J. Comput. Syst. Sci.* 60, 3 (2000), 630–659.
- [8] Flavio Chierichetti, Ravi Kumar, Silvio Lattanzi, Michael Mitzenmacher, Alessandro Panconesi, and Prabhakar Raghavan. 2009. On Compressing Social Networks. In *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD 2009, Paris, France, June 28 - July 1, 2009*. ACM, 219–228.
- [9] Laxman Dhulipala, Igor Kabiljo, Brian Karrer, Giuseppe Ottaviano, Sergey Pupyrev, and Alon Shalita. 2016. Compressing Graphs and Indexes with Recursive Graph Bisection. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD 2016, San Francisco, CA, USA, August 13-17, 2016*. ACM, 1535–1544.
- [10] Harold N. Gabow. 1983. An Efficient Reduction Technique for Degree-Constrained Subgraph and Bidirected Network Flow Problems. In *Proceedings of the 15th Annual ACM Symposium on Theory of Computing, STOC 1983, Boston, MA, USA, April 25-27, 1983*. ACM, 448–456.
- [11] Kifayat-Ullah Khan, Waqas Nawaz, and Young-Koo Lee. 2015. Set-based Approximate Approach for Lossless Graph Summarization. *Computing* 97, 12 (2015), 1185–1207.
- [12] Jihoon Ko, Yunbum Kook, and Kijung Shin. 2020. Incremental Lossless Graph Summarization. In *KDD '20: The 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Virtual Event, CA, USA, August 23-27, 2020*, Rajesh Gupta, Yan Liu, Jiliang Tang, and B. Aditya Prakash (Eds.). ACM, 317–327.
- [13] Kyuhan Lee, Hyeonsoo Jo, Jihoon Ko, Sungsu Lim, and Kijung Shin. 2020. SSumM: Sparse Summarization of Massive Graphs. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (Virtual Event, CA, USA) (KDD '20)*. Association for Computing Machinery, New York, NY, USA, 144–154.
- [14] Kyuhan Lee, Jihoon Ko, and Kijung Shin. 2022. SLUGGER: Lossless Hierarchical Summarization of Massive Graphs. In *38th IEEE International Conference on Data Engineering, ICDE 2022, Kuala Lumpur, Malaysia, May 9-12, 2022*. IEEE, 472–484.
- [15] Jure Leskovec and Andrej Krevl. 2014. SNAP Datasets: Stanford Large Network Dataset Collection. <http://snap.stanford.edu/data>. Accessed for benchmark graph datasets.
- [16] Yongsub Lim, U Kang, and Christos Faloutsos. 2014. Slashburn: Graph compression and mining beyond caveman communities. *IEEE Transactions on Knowledge and Data Engineering* 26, 12 (2014), 3077–3089.
- [17] Yike Liu, Tara Safavi, Abhilash Dighe, and Danai Koutra. 2018. Graph Summarization Methods and Applications: A Survey. *ACM Comput. Surv.* 51, 3 (2018), 62:1–62:34.
- [18] Duane Merrill, Michael Garland, and Andrew S. Grimshaw. 2012. Scalable GPU graph traversal. In *Proceedings of the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPOPP 2012, New Orleans, LA, USA, February 25-29, 2012*, J. Ramanujam and P. Sadayappan (Eds.). ACM, 117–128.
- [19] Saket Navlakha, Rajeev Rastogi, and Nisheeth Shrivastava. 2008. Graph summarization with bounded error. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2008, Vancouver, BC, Canada, June 10-12, 2008*, Jason Tsong-Li Wang (Ed.). ACM, 419–432.
- [20] Hojin Seo, Kisung Park, Yongkoo Han, Hyunwook Kim, Muhammad Umair, Kifayat Ullah Khan, and Young Koo Lee. 2020. An Effective Graph Summarization and Compression Technique for a Large-scaled Graph. *J. Supercomput.* 76, 10 (2020), 7906–7920.
- [21] Kijung Shin, Amol Ghoting, Myunghwan Kim, and Hema Raghavan. 2019. SWeG: Lossless and Lossy Summarization of Web-Scale Graphs. In *The World Wide Web Conference, WWW 2019, San Francisco, CA, USA, May 13-17, 2019*, Ling Liu, Ryan W. White, Amin Mantrach, Fabrizio Silvestri, Julian J. McAuley, Ricardo Baeza-Yates, and Leila Zia (Eds.). ACM, 1679–1690.
- [22] Julian Shun and Guy E. Blelloch. 2013. Lagra: a lightweight graph processing framework for shared memory. In *ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPOPP '13, Shenzhen, China, February 23-27, 2013*, Alex Nicolau, Xiaowei Shen, Saman P. Amarasinghe, and Richard W. Vuduc (Eds.). ACM, 135–146.
- [23] Julian Shun, Laxman Dhulipala, and Guy E. Blelloch. 2015. Smaller and Faster: Parallel Processing of Compressed Graphs with Lagra+. In *2015 Data Compression Conference, DCC 2015, Snowbird, UT, USA, April 7-9, 2015*. IEEE, 403–412.
- [24] Yangzihao Wang, Andrew A. Davidson, Yuechao Pan, Yuduo Wu, Andy Riffel, and John D. Owens. 2016. Gunrock: a high-performance graph processing library on the GPU. In *Proceedings of the 21st ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPOPP 2016, Barcelona, Spain, March 12-16, 2016*, Rafael Asenjo and Tim Harris (Eds.). ACM, 11:1–11:12.
- [25] Rui Yan, Zi Yuan, Xiaojun Wan, Yan Zhang, and Xiaoming Li. 2012. Hierarchical Graph Summarization: Leveraging Hybrid Information through Visible and Invisible Linkage. In *Advances in Knowledge Discovery and Data Mining, Pang-Ning Tan, Sanjay Chawla, Chin Kuan Ho, and James Bailey (Eds.)*. Springer Berlin Heidelberg, Berlin, Heidelberg, 97–108.
- [26] Carl Yang, Aydin Buluç, and John D. Owens. 2022. Graph-BLAST: A High-Performance Linear Algebra-based Graph Framework on the GPU. *ACM Trans. Math. Softw.* 48, 1 (2022), 1:1–1:51.
- [27] Quinton Yong, Mahdi Hajiabadi, Venkatesh Srinivasan, and Alex Thomo. 2021. Efficient Graph Summarization using Weighted LSH at Billion-Scale (*SIGMOD '21*). Association for Computing Machinery, New York, NY, USA, 2357–2365.